Straep — Tech Stack and Implementation Guide

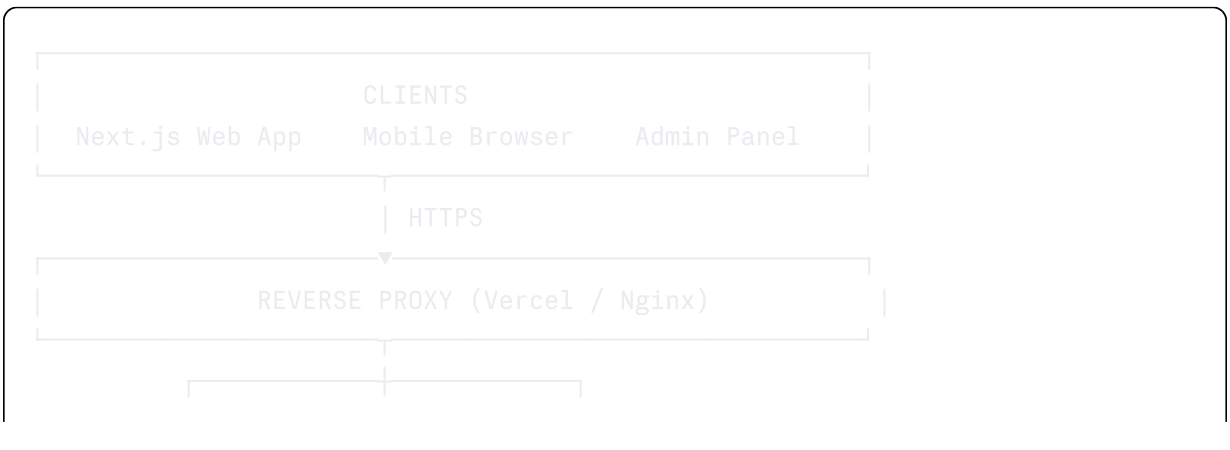
Version: 1.0
Document owner: Engineering Team
Purpose: Definitive guide to every technology choice, configuration decision, and implementation pattern used in building Straep. Engineers should read this before writing a single line of code.

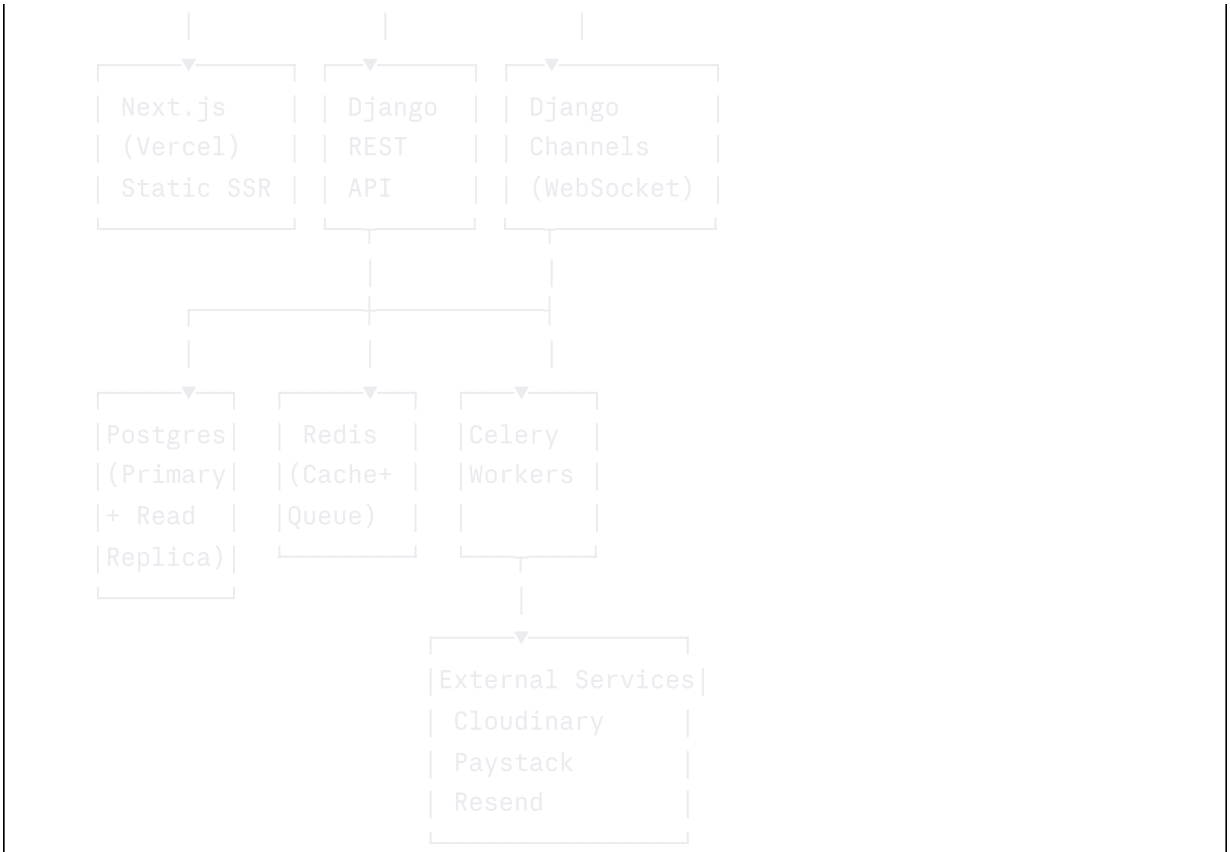
Table of Contents

---

1. Architecture Overview

1.1 System Architecture Diagram





1.2 Key Architectural Decisions

Decision	Choice	Rationale
Frontend framework	Next.js 14 (App Router)	SSR for SEO on public pages; RSC for performance
Backend framework	Django + DRF	Rapid development; robust ORM; Python ecosystem
Database	PostgreSQL 16	JSONB for flexible page sections; excellent performance; pgvector ready for Phase 5
Cache layer	Redis 7	Feed storage, session data, rate limiting, task queuing
Task queue	Celery + Redis broker	Background jobs: email, feed fan-out, analytics aggregation
Media	Cloudinary	Automatic image optimisation, CDN delivery, signed upload presets
Auth	JWT (access + refresh)	Stateless API; mobile-compatible; refresh rotation for security
Payments (primary)	Paystack	Nigeria-first; local card support; excellent webhook reliability
Payments (global)	Stripe (Phase 2)	International card processing

Decision	Choice	Rationale
Email	Resend	Modern SMTP; excellent deliverability; React Email templates
Real-time	Django Channels	WebSocket support for notifications and messaging (Phase 3)
Hosting (frontend)	Vercel	Zero-config Next.js deployment; edge CDN
Hosting (backend)	Railway or Render	Simple Django + Celery + Redis deployment

2. Frontend — Next.js

2.1 Technology Stack

```
Next.js 14 (App Router)
TypeScript 5
Tailwind CSS 3
TanStack Query (React Query) v5
Zustand (global state)
React Hook Form + Zod (forms and validation)
Framer Motion (animations)
Lucide React (icons)
next/image (image optimisation)
next-themes (dark/light mode — Phase 2)
```

2.2 Next.js App Router Structure

```
apps/web/
├── app/
│   ├── layout.tsx                    ← Root layout (providers, fonts)
│   ├── page.tsx                      ← Landing page (/)
│   │
│   ├── (marketing)/                  ← Route group: public marketing pages
│   │   ├── about/page.tsx
│   │   ├── features/page.tsx
│   │   ├── for-businesses/page.tsx
│   │   ├── for-creators/page.tsx
│   │   ├── for-developers/page.tsx
│   │   ├── marketplace/page.tsx
│   │   ├── pricing/page.tsx
│   │   └── blog/
│   │       ├── page.tsx
│   │       └── [slug]/page.tsx
│   │
│   ├── (auth)/                       ← Route group: auth pages (no shared
layout)
```

```
│   │   ├── layout.tsx
│   │   ├── get-started/page.tsx
│   │   ├── login/page.tsx
│   │   ├── forgot-password/page.tsx
│   │   ├── reset-password/page.tsx
│   │   ├── verify-email/page.tsx
│   │   └── onboarding/
│   │       ├── account-type/page.tsx
│   │       ├── personal-setup/page.tsx
│   │       ├── business-setup/page.tsx
│   │       ├── creator-setup/page.tsx
│   │       ├── interests/page.tsx
│   │       ├── follow-suggestions/page.tsx
│   │       └── complete/page.tsx
│   │
│   ├── (app)/                        ← Route group: logged-in app (shared
layout)
│   │   ├── layout.tsx                ← App shell (navbar, bottom nav)
│   │   ├── home/page.tsx
│   │   ├── discover/
│   │   │   ├── page.tsx
│   │   │   ├── search/page.tsx
│   │   │   ├── category/[slug]/page.tsx
│   │   │   └── location/[city]/page.tsx
│   │   ├── market/
│   │   │   ├── page.tsx
│   │   │   ├── product/[id]/page.tsx
│   │   │   └── service/[id]/page.tsx
│   │   ├── communities/
│   │   │   ├── page.tsx
│   │   │   └── [slug]/page.tsx
│   │   ├── create/
│   │   │   └── [[...type]]/page.tsx
│   │   ├── messages/
│   │   │   ├── page.tsx
│   │   │   └── [conversationId]/page.tsx
│   │   ├── notifications/page.tsx
│   │   └── saved/page.tsx
│   │
│   ├── p/                            ← Public brand pages
│   │   └── [username]/
│   │       ├── page.tsx              ← Brand page (SSR, cached)
│   │       ├── products/page.tsx
│   │       ├── services/page.tsx
│   │       ├── posts/page.tsx
│   │       └── reviews/page.tsx
│   │
│   ├── dashboard/                    ← Business dashboard (auth + type
check)
│   │   ├── layout.tsx
│   │   ├── page.tsx                  ← Overview
│   │   ├── brand-page/
```

```
│   │   │   ├── page.tsx
│   │   │   └── editor/page.tsx
│   │   ├── storefront/
│   │   │   ├── page.tsx
│   │   │   ├── products/
│   │   │   │   ├── page.tsx
│   │   │   │   ├── add/page.tsx
│   │   │   │   └── [id]/edit/page.tsx
│   │   │   └── services/
│   │   │       ├── page.tsx
│   │   │       ├── add/page.tsx
│   │   │       └── [id]/edit/page.tsx
│   │   ├── leads/
│   │   │   ├── page.tsx
│   │   │   └── [id]/page.tsx
│   │   ├── analytics/page.tsx
│   │   ├── templates/
│   │   │   ├── page.tsx
│   │   │   └── marketplace/page.tsx
│   │   └── subscription/
│   │       ├── page.tsx
│   │       ├── upgrade/page.tsx
│   │       └── success/page.tsx
│   │
│   ├── dev/                          ← Developer dashboard
│   │   ├── layout.tsx
│   │   ├── page.tsx
│   │   ├── templates/page.tsx
│   │   ├── upload/page.tsx
│   │   ├── sales/page.tsx
│   │   ├── payouts/page.tsx
│   │   └── settings/page.tsx
│   │
│   └── admin/                        ← Admin panel
│       ├── layout.tsx
│       ├── page.tsx
│       ├── users/
│       │   ├── page.tsx
│       │   └── [id]/page.tsx
│       ├── brands/page.tsx
│       ├── templates/
│       │   ├── page.tsx
│       │   └── [id]/page.tsx
│       ├── reports/page.tsx
│       └── verification/page.tsx
│
├── components/
│   ├── ui/                           ← Primitive UI components (Button,
Input, Modal, etc.)
│   ├── brand/                        ← Brand-specific components (BrandCard,
BrandPageSection)
│   ├── feed/                         ← Feed components (PostCard,
```

```
FeedFilter)
|   ├── storefront/                ← Storefront components (ProductCard,
ProductModal)
|   ├── inquiry/                   ← Inquiry modal and form
|   ├── dashboard/                 ← Dashboard-specific components
|   ├── editor/                    ← Brand page editor components
|   └── layouts/                   ← Layout wrappers (AppShell,
DashboardShell)
|
├── lib/
|   ├── api.ts                     ← Axios instance with auth interceptors
|   ├── auth.ts                    ← Auth utilities (token storage,
decode)
|   ├── constants.ts               ← App-wide constants
|   ├── utils.ts                   ← Helper functions
|   └── validations/               ← Zod schemas
|
├── hooks/
|   ├── useAuth.ts                 ← Authentication hook
|   ├── usePlan.ts                 ← Plan features hook (check plan gates)
|   ├── useFeed.ts                 ← Feed query hook
|   └── useBrandPage.ts            ← Brand page query hook
|
└── stores/
    ├── authStore.ts               ← Zustand: user + plan state
    └── editorStore.ts             ← Zustand: brand page editor state
```

2.3 API Client Configuration

typescript

```typescript
// lib/api.ts
import axios from 'axios';

const api = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL,
  withCredentials: true,  // Send cookies (refresh token)
  headers: { 'Content-Type': 'application/json' },
});

// Request interceptor: attach access token
api.interceptors.request.use((config) => {
  const token = getAccessToken();  // from Zustand store / memory
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Response interceptor: handle token refresh on 401
api.interceptors.response.use(
  (response) => response,
  async (error) => {
    if (error.response?.status === 401 && !error.config._retry) {
      error.config._retry = true;
      try {
        const { data } = await axios.post('/auth/refresh', {}, { withCredential
        setAccessToken(data.data.access_token);  // update store
        return api(error.config);  // retry original request
      } catch {
        clearAuth();  // logout
        window.location.href = '/auth/login';
      }
    }
    return Promise.reject(error);
  }
);

export default api;
```

2.4 Plan Gate Hook

typescript

```
// hooks/usePlan.ts
import { useAuthStore } from '@/stores/authStore';

type PlanFeature = keyof PlanFeatures;

export function usePlan() {
  const planFeatures = useAuthStore((s) => s.planFeatures);
  const plan = useAuthStore((s) => s.subscription?.plan ?? 'free');

  function can(feature: PlanFeature): boolean {
    return planFeatures?.[feature] === true;
  }

  function requirePlan(feature: PlanFeature, minPlan: string): boolean {
    return can(feature);
  }

  return { plan, can, planFeatures };
}

// Usage in a component:
// const { can } = usePlan();
// if (!can('full_customisation')) { showUpgradeModal('custom_brand_colours');
```

2.5 Brand Page Editor State (Zustand)

typescript

```typescript
// stores/editorStore.ts
import { create } from 'zustand';

interface Section {
  id: string;
  type: string;
  visible: boolean;
  order: number;
  settings: Record<string, unknown>;
}

interface EditorStore {
  sections: Section[];
  isDirty: boolean;
  lastSaved: Date | null;

  updateSection: (id: string, settings: Partial<Section['settings']>) => void;
  reorderSections: (newOrder: Section[]) => void;
  toggleSectionVisibility: (id: string) => void;
  addSection: (type: string) => void;
  removeSection: (id: string) => void;
  markSaved: () => void;
}

export const useEditorStore = create<EditorStore>((set) => ({
  sections: [],
  isDirty: false,
  lastSaved: null,

  updateSection: (id, settings) =>
    set((state) => ({
      sections: state.sections.map((s) =>
        s.id === id ? { ...s, settings: { ...s.settings, ...settings } } : s
      ),
      isDirty: true,
    })),

  reorderSections: (newOrder) =>
    set({ sections: newOrder.map((s, i) => ({ ...s, order: i })), isDirty: true

  markSaved: () => set({ isDirty: false, lastSaved: new Date() }),
}));
```

2.6 SSR for Brand Pages

Brand pages use Next.js `generateStaticParams` for the most popular pages and ISR (Incremental Static Regeneration) for the rest:

typescript

```tsx
// app/p/[username]/page.tsx
import { Metadata } from 'next';

export const revalidate = 300; // Revalidate every 5 minutes (ISR)

export async function generateMetadata({ params }): Promise<Metadata> {
  const brand = await fetchBrandPage(params.username);
  return {
    title: `${brand.business_name} on Straep`,
    description: brand.description,
    openGraph: {
      title: brand.business_name,
      description: brand.description,
      images: [brand.logo_url || brand.cover_url],
    },
  };
}

export default async function BrandPage({ params }) {
  const brand = await fetchBrandPage(params.username);
  if (!brand) notFound();
  return <BrandPageView brand={brand} />;
}
```

---

3. Backend — Django

3.1 Django Project Structure

```
apps/backend/
├── config/
│   ├── settings/
│   │   ├── base.py           ← Shared settings
│   │   ├── development.py    ← Dev overrides
│   │   └── production.py     ← Prod overrides
│   ├── urls.py               ← Root URL config
│   ├── wsgi.py
│   └── asgi.py               ← For Django Channels
│
├── apps/
│   ├── users/                ← Custom User model, auth, profile
│   │   ├── models.py
│   │   ├── serializers.py
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── permissions.py    ← Custom permission classes
│   │   └── services.py       ← Business logic layer
│   │
│   ├── brands/               ← BrandPage and editor
│   │   ├── models.py
```

```
│   │   ├── serializers.py
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── services.py
│   │   └── cache.py          ← Redis cache operations
│   │
│   ├── storefront/           ← Products and services
│   ├── inquiries/            ← Inquiry and lead management
│   ├── posts/                ← Post, comment, poll
│   ├── feed/                 ← Feed generation and caching
│   ├── social/               ← Follows, likes, saves
│   ├── subscriptions/        ← Plans, subscriptions, webhooks
│   ├── templates_market/     ← Template marketplace
│   ├── notifications/        ← Notification system
│   ├── messaging/            ← Django Channels messaging (Phase 3)
│   ├── reviews/              ← Review system (Phase 3)
│   ├── communities/          ← Community features (Phase 3)
│   ├── admin_panel/          ← Admin API endpoints
│   └── analytics/            ← Analytics data and aggregation
│
├── common/
│   ├── mixins.py             ← Reusable view mixins
│   ├── pagination.py         ← Custom pagination classes
│   ├── permissions.py        ← Platform-wide permissions
│   ├── exceptions.py         ← Custom exception handlers
│   └── utils.py              ← Utility functions
│
└── tasks/
    ├── email.py              ← Email Celery tasks
    ├── feed.py               ← Feed fan-out tasks
    ├── analytics.py          ← Daily analytics aggregation
    ├── notifications.py      ← Notification delivery tasks
    └── payments.py           ← Payment processing tasks
```

3.2 Django Settings (Key Configuration)

python

```python
# config/settings/base.py

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # Third party
    'rest_framework',
    'corsheaders',
    'django_filters',
    'channels',

    # Local apps
    'apps.users',
    'apps.brands',
    'apps.storefront',
    'apps.inquiries',
    'apps.posts',
    'apps.feed',
    'apps.social',
    'apps.subscriptions',
    'apps.templates_market',
    'apps.notifications',
    'apps.admin_panel',
    'apps.analytics',
]

AUTH_USER_MODEL = 'users.User'

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'apps.users.authentication.JWTAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
    'DEFAULT_PAGINATION_CLASS': 'common.pagination.StandardPagination',
    'PAGE_SIZE': 20,
    'DEFAULT_FILTER_BACKENDS': [
        'django_filters.rest_framework.DjangoFilterBackend',
        'rest_framework.filters.SearchFilter',
        'rest_framework.filters.OrderingFilter',
    ],
    'EXCEPTION_HANDLER': 'common.exceptions.custom_exception_handler',
}

# Database
```

```python
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': env('DB_NAME'),
        'USER': env('DB_USER'),
        'PASSWORD': env('DB_PASSWORD'),
        'HOST': env('DB_HOST'),
        'PORT': env('DB_PORT', default='5432'),
        'CONN_MAX_AGE': 60,
        'OPTIONS': {
            'connect_timeout': 10,
        },
    }
}

# Redis
REDIS_URL = env('REDIS_URL', default='redis://localhost:6379/0')

# Cache
CACHES = {
    'default': {
        'BACKEND': 'django.core.cache.backends.redis.RedisCache',
        'LOCATION': REDIS_URL,
        'OPTIONS': {
            'CLIENT_CLASS': 'django_redis.client.DefaultClient',
        },
        'KEY_PREFIX': 'straep',
        'TIMEOUT': 300,  # 5 minutes default
    }
}

# Celery
CELERY_BROKER_URL = REDIS_URL
CELERY_RESULT_BACKEND = REDIS_URL
CELERY_ACCEPT_CONTENT = ['json']
CELERY_TASK_SERIALIZER = 'json'
CELERY_RESULT_SERIALIZER = 'json'
CELERY_TIMEZONE = 'Africa/Lagos'
CELERY_BEAT_SCHEDULER = 'django_celery_beat.schedulers:DatabaseScheduler'

# JWT
JWT_ACCESS_TOKEN_EXPIRES = 15 * 60           # 15 minutes
JWT_REFRESH_TOKEN_EXPIRES = 30 * 24 * 3600   # 30 days
JWT_ALGORITHM = 'HS256'
JWT_SECRET_KEY = env('JWT_SECRET_KEY')

# Cloudinary
CLOUDINARY_CLOUD_NAME = env('CLOUDINARY_CLOUD_NAME')
CLOUDINARY_API_KEY = env('CLOUDINARY_API_KEY')
CLOUDINARY_API_SECRET = env('CLOUDINARY_API_SECRET')

# Paystack
```

```python
PAYSTACK_SECRET_KEY = env('PAYSTACK_SECRET_KEY')
PAYSTACK_PUBLIC_KEY = env('PAYSTACK_PUBLIC_KEY')

# Resend (email)
RESEND_API_KEY = env('RESEND_API_KEY')
DEFAULT_FROM_EMAIL = 'Straep <hello@straep.com>'

# CORS
CORS_ALLOWED_ORIGINS = [
    'https://straep.com',
    'https://www.straep.com',
]
CORS_ALLOW_CREDENTIALS = True  # Required for cookies
```

3.3 Custom JWT Authentication

python

```python
# apps/users/authentication.py
import jwt
from django.conf import settings
from rest_framework.authentication import BaseAuthentication
from rest_framework.exceptions import AuthenticationFailed
from apps.users.models import User

class JWTAuthentication(BaseAuthentication):
    def authenticate(self, request):
        auth_header = request.headers.get('Authorization')
        if not auth_header or not auth_header.startswith('Bearer '):
            return None

        token = auth_header.split(' ')[1]
        try:
            payload = jwt.decode(
                token,
                settings.JWT_SECRET_KEY,
                algorithms=[settings.JWT_ALGORITHM]
            )
        except jwt.ExpiredSignatureError:
            raise AuthenticationFailed({
                'code': 'TOKEN_EXPIRED',
                'message': 'Access token has expired.'
            })
        except jwt.InvalidTokenError:
            raise AuthenticationFailed({
                'code': 'INVALID_TOKEN',
                'message': 'Invalid access token.'
            })

        try:
            user = User.objects.select_related('profile', 'subscription__plan')
                id=payload['user_id'],
                is_active=True,
                deleted_at__isnull=True
            )
        except User.DoesNotExist:
            raise AuthenticationFailed({'code': 'USER_NOT_FOUND'})

        return (user, token)
```

3.4 Plan Permission Middleware

```python
```

```python
# apps/subscriptions/permissions.py
from rest_framework.permissions import BasePermission
from rest_framework.exceptions import PermissionDenied

def require_plan_feature(feature_name):
    """
    Returns a DRF permission class that checks for a specific plan feature.

    Usage:
        permission_classes = [IsAuthenticated, require_plan_feature('full_custo
    """
    class PlanFeaturePermission(BasePermission):
        def has_permission(self, request, view):
            if not request.user.is_authenticated:
                return False

            subscription = getattr(request.user, 'subscription', None)
            if not subscription or not subscription.plan:
                # Free plan
                plan_features = get_free_plan_features()
            else:
                plan_features = subscription.plan.features

            if not plan_features.get(feature_name, False):
                raise PermissionDenied({
                    'code': 'PLAN_REQUIRED',
                    'message': f'This feature requires a higher plan.',
                    'details': {
                        'feature': feature_name,
                        'required_plan': get_required_plan_for_feature(feature_
                        'current_plan': getattr(subscription, 'plan', {}).get('
                    }
                })
            return True

    PlanFeaturePermission.__name__ = f'PlanFeature_{feature_name}'
    return PlanFeaturePermission
```

---

4. Database — PostgreSQL

4.1 Connection Pooling

Use **PgBouncer** in transaction mode for production. This is critical under load because Django opens a new connection per request.

```
PostgreSQL Config (postgresql.conf):
  max_connections = 100        (PgBouncer connects to this)

PgBouncer Config (pgbouncer.ini):
  pool_mode = transaction
  max_client_conn = 1000
  default_pool_size = 25
```

Railway and Render provide PgBouncer-compatible connection pooling by default.

4.2 Read Replica

For Phase 2 and beyond, add a read replica. Route all `GET` API calls (feed, discover, brand page) to the replica; route all writes to the primary.

python

```python
# config/settings/production.py
DATABASES = {
    'default': { ... primary ... },
    'replica': { ... replica ... }
}

DATABASE_ROUTERS = ['common.db_routers.PrimaryReplicaRouter']
```

4.3 JSONB Queries

The `sections_json` field on `BrandPage` is stored as JSONB. When querying or filtering by section content:

python

```python
# Example: find all brand pages with a "testimonials" section
from django.db.models import Q

BrandPage.objects.filter(
    sections_json__contains=[{"type": "testimonials"}]
)
```

---

5. Caching — Redis

5.1 Cache Key Strategy

All Redis keys follow the pattern: `straep:{entity}:{id}:{descriptor}`

Cache Key	TTL	Content	Invalidation
`straep:brand_page:{slug}`	5 min	Full serialised brand page JSON	On publish or brand update
`straep:feed:{user_id}`	5 min	Sorted set of post IDs	On new post by followed brand
`straep:plan_features:{user_id}`	30 min	Plan features dict	On subscription change
`straep:categories`	1 hour	Category list	On category change (admin)
`straep:discover_featured`	1 hour	Featured brands list	Admin-triggered
`straep:notifications:unread:{user_id}`	5 min	Unread count	On new notification
`straep:rate_limit:inquiry:{ip}:{brand_id}`	24 hours	Request count	Auto-expire

5.2 Feed Cache Design

The feed is stored in Redis as a sorted set per user:

```
Key: straep:feed:{user_id}
Type: Sorted Set (ZSET)
Score: Unix timestamp of the post (for chronological ordering)
Member: post_id (UUID string)
```

Fan-out on write (for brands with < 10,000 followers):

```python
```

```python
# tasks/feed.py
@celery_app.task
def fan_out_post_to_followers(post_id: str, author_brand_id: str):
    """Push a new post to all followers' feed caches."""
    follower_ids = UserFollow.objects.filter(
        brand_page_id=author_brand_id
    ).values_list('follower_id', flat=True)

    post = Post.objects.get(id=post_id)
    timestamp = post.created_at.timestamp()

    pipeline = redis_client.pipeline()
    for user_id in follower_ids:
        key = f'straep:feed:{user_id}'
        pipeline.zadd(key, {post_id: timestamp})
        pipeline.zremrangebyrank(key, 0, -501)  # Keep max 500 posts per user f
    pipeline.execute()
```

Feed read:

python

```python
def get_user_feed(user_id: str, cursor: str | None, limit: int = 20):
    key = f'straep:feed:{user_id}'

    if cursor:
        # Get posts older than the cursor timestamp
        max_score = float(cursor)
        post_ids = redis_client.zrevrangebyscore(key, max_score - 1, '-inf', st
    else:
        post_ids = redis_client.zrevrange(key, 0, limit)

    posts = Post.objects.filter(
        id__in=post_ids,
        is_published=True,
        is_removed=False,
        deleted_at__isnull=True
    ).select_related('author', 'brand_page', 'product').order_by('-created_at')

    has_more = len(post_ids) > limit
    next_cursor = posts[-1].created_at.timestamp() if has_more else None

    return posts[:limit], next_cursor, has_more
```

---

6. Background Tasks — Celery

6.1 Task Definitions

```python
```

```python
# tasks/email.py
@celery_app.task(bind=True, max_retries=3, default_retry_delay=60)
def send_inquiry_notification_email(self, inquiry_id: str):
    try:
        inquiry = Inquiry.objects.select_related('brand_page__user').get(id=inq
        brand_email = inquiry.brand_page.contact_email or inquiry.brand_page.us

        send_email(
            to=brand_email,
            subject=f'New inquiry from {inquiry.sender_name}',
            template='emails/new_inquiry.html',
            context={
                'inquiry': inquiry,
                'brand': inquiry.brand_page,
                'dashboard_url': f'https://straep.com/dashboard/leads/{inquiry.
            }
        )
    except Exception as exc:
        raise self.retry(exc=exc)

# tasks/analytics.py
@celery_app.task
def aggregate_daily_brand_stats():
    """Run nightly. Aggregates raw view events into daily_stats table."""
    yesterday = date.today() - timedelta(days=1)

    brands = BrandPage.objects.filter(is_published=True)
    for brand in brands:
        view_count = BrandPageView.objects.filter(
            brand_page=brand,
            viewed_at__date=yesterday
        ).count()

        unique_visitors = BrandPageView.objects.filter(
            brand_page=brand,
            viewed_at__date=yesterday
        ).values('ip_address').distinct().count()

        BrandPageViewDailyStats.objects.update_or_create(
            brand_page=brand,
            date=yesterday,
            defaults={
                'view_count': view_count,
                'unique_visitors': unique_visitors,
            }
        )
```

6.2 Celery Beat Schedule

python

```python
# config/settings/base.py
from celery.schedules import crontab

CELERY_BEAT_SCHEDULE = {
    'aggregate-daily-brand-stats': {
        'task': 'tasks.analytics.aggregate_daily_brand_stats',
        'schedule': crontab(hour=1, minute=0),  # 1:00 AM WAT daily
    },
    'check-expired-subscriptions': {
        'task': 'tasks.payments.check_expired_subscriptions',
        'schedule': crontab(hour=0, minute=30),  # 12:30 AM WAT daily
    },
    'send-renewal-reminders': {
        'task': 'tasks.payments.send_renewal_reminder_emails',
        'schedule': crontab(hour=9, minute=0),  # 9:00 AM WAT daily
    },
    'clean-expired-otps': {
        'task': 'tasks.auth.clean_expired_otps',
        'schedule': crontab(minute=0, hour='*/6'),  # Every 6 hours
    },
}
```

---

7. Media Storage — Cloudinary

7.1 Upload Strategy

Use **client-side direct upload** to Cloudinary. The Django backend generates signed upload parameters; the client uploads directly; the client sends the resulting URL back to the API.

```python
```

```python
# apps/brands/views.py
import cloudinary
from cloudinary.utils import cloudinary_url, private_download_url
import time, hashlib, hmac

def get_upload_params(upload_type: str, user_id: str) -> dict:
    timestamp = int(time.time())

    # Define folder and transformation per upload type
    configs = {
        'logo': {'folder': f'brands/{user_id}/logo', 'width': 400, 'height': 40
        'cover': {'folder': f'brands/{user_id}/cover', 'width': 1600, 'height':
        'product': {'folder': f'brands/{user_id}/products', 'width': 800, 'crop
        'avatar': {'folder': f'users/{user_id}/avatar', 'width': 200, 'height':
        'post': {'folder': f'brands/{user_id}/posts', 'width': 1200, 'crop': 'l
    }

    config = configs.get(upload_type, configs['product'])

    params = {
        'timestamp': timestamp,
        'folder': config['folder'],
        'upload_preset': f'straep_{upload_type}',
        'api_key': settings.CLOUDINARY_API_KEY,
    }

    # Generate signature
    param_string = '&'.join(f'{k}={v}' for k, v in sorted(params.items()))
    signature = hashlib.sha1(
        f'{param_string}{settings.CLOUDINARY_API_SECRET}'.encode()
    ).hexdigest()

    params['signature'] = signature
    params['cloud_name'] = settings.CLOUDINARY_CLOUD_NAME
    params['upload_url'] = f'https://api.cloudinary.com/v1_1/{settings.CLOUDINA

    return params
```

7.2 Cloudinary Upload Presets

Configure these presets in the Cloudinary dashboard:

Preset	Max File Size	Transformations	Folder
`straep_logo`	5MB	Auto-format, auto-quality, 400×400	`brands/:user_id/logo`
`straep_cover`	10MB	Auto-format, auto-quality, 1600×400	`brands/:user_id/cover`
`straep_product`	8MB	Auto-format, auto-quality, max 1200px	`brands/:user_id/products`
`straep_avatar`	5MB	Auto-format, auto-quality, 200×200	`users/:user_id/avatar`
`straep_post`	10MB	Auto-format, auto-quality, max 1200px	`brands/:user_id/posts`
`straep_template_preview`	8MB	Auto-format, auto-quality, max 1400px	`templates/previews`

8. Authentication — JWT

8.1 Token Structure

Access Token Payload:

```json
{
  "user_id": "550e8400-e29b-41d4-a716-446655440000",
  "email": "user@example.com",
  "account_type": "business",
  "iat": 1707567890,
  "exp": 1707568790
}
```

Refresh Token: Random 64-byte token, stored as SHA-256 hash in the database. Never store the plaintext refresh token in the database.

8.2 Token Storage on the Client

Token	Storage	Notes
Access token	JavaScript memory (Zustand store)	Never localStorage; cleared on page close

Token	Storage	Notes
Refresh token	HTTP-only cookie	Set by the server; not accessible to JS

typescript

```typescript
// stores/authStore.ts
import { create } from 'zustand';

interface AuthStore {
  accessToken: string | null;
  user: User | null;
  planFeatures: PlanFeatures | null;
  setAuth: (token: string, user: User) => void;
  clearAuth: () => void;
}

export const useAuthStore = create<AuthStore>((set) => ({
  accessToken: null,
  user: null,
  planFeatures: null,
  setAuth: (token, user) => set({ accessToken: token, user, planFeatures: user.
  clearAuth: () => set({ accessToken: null, user: null, planFeatures: null }),
}));
```

9. Payments — Paystack and Stripe

9.1 Paystack Integration

python

```python
# apps/subscriptions/services/paystack.py
import requests
from django.conf import settings

PAYSTACK_BASE_URL = 'https://api.paystack.co'

def initialize_transaction(email: str, amount_kobo: int, reference: str, metada
    """Returns the authorization_url for the checkout page."""
    response = requests.post(
        f'{PAYSTACK_BASE_URL}/transaction/initialize',
        headers={'Authorization': f'Bearer {settings.PAYSTACK_SECRET_KEY}'},
        json={
            'email': email,
            'amount': amount_kobo,
            'reference': reference,
            'callback_url': f'https://straep.com/dashboard/subscription/callbac
            'metadata': metadata,
        }
    )
    response.raise_for_status()
    data = response.json()
    return data['data']['authorization_url']

def verify_transaction(reference: str) -> dict:
    """Verify a transaction after webhook or callback."""
    response = requests.get(
        f'{PAYSTACK_BASE_URL}/transaction/verify/{reference}',
        headers={'Authorization': f'Bearer {settings.PAYSTACK_SECRET_KEY}'},
    )
    response.raise_for_status()
    return response.json()['data']
```

9.2 Webhook Handler

python

```python
# apps/subscriptions/views.py
import hmac
import hashlib
import json
from django.http import HttpResponse
from django.views.decorators.csrf import csrf_exempt

@csrf_exempt
def paystack_webhook(request):
    if request.method != 'POST':
        return HttpResponse(status=405)

    # Verify signature
    payload = request.body
    signature = request.headers.get('X-Paystack-Signature', '')
    expected = hmac.new(
        settings.PAYSTACK_SECRET_KEY.encode(),
        payload,
        hashlib.sha512
    ).hexdigest()

    if not hmac.compare_digest(signature, expected):
        return HttpResponse(status=401)

    event = json.loads(payload)
    event_type = event.get('event')

    # Handle events
    handlers = {
        'charge.success': handle_charge_success,
        'subscription.disable': handle_subscription_disable,
        'invoice.payment_failed': handle_payment_failed,
        'transfer.success': handle_transfer_success,
    }

    handler = handlers.get(event_type)
    if handler:
        # Run async to respond to Paystack immediately
        handle_webhook_event.delay(event_type, event['data'])

    return HttpResponse(status=200)  # Always return 200 quickly
```

---

10. Email — Resend

10.1 Email Service

python

```python
# common/email.py
import resend
from django.conf import settings
from django.template.loader import render_to_string

resend.api_key = settings.RESEND_API_KEY

def send_email(to: str | list, subject: str, template: str, context: dict):
    html = render_to_string(template, context)

    resend.Emails.send({
        'from': settings.DEFAULT_FROM_EMAIL,
        'to': to if isinstance(to, list) else [to],
        'subject': subject,
        'html': html,
    })
```

10.2 Email Templates

All email templates are in `templates/emails/`. Base template includes header (Straep logo) and footer (unsubscribe link, address).

Template	Trigger
`new_inquiry.html`	New inquiry received
`otp_verification.html`	Registration OTP
`password_reset.html`	Password reset link
`subscription_activated.html`	Successful upgrade
`subscription_cancelled.html`	Cancellation confirmation
`payment_failed.html`	Failed payment
`renewal_reminder.html`	7 days before renewal
`template_approved.html`	Developer template approved
`template_rejected.html`	Developer template rejected
`verification_approved.html`	Business verification approved

---

11. Real-Time — Django Channels

Phase 3 — not required for MVP. The WebSocket infrastructure should be set up from Phase 1 so it doesn't require a significant refactor.

11.1 ASGI Configuration

python

```python
# config/asgi.py
import os
from django.core.asgi import get_asgi_application
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.middleware import BaseMiddleware
from apps.messaging.routing import websocket_urlpatterns

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings.production')

application = ProtocolTypeRouter({
    'http': get_asgi_application(),
    'websocket': URLRouter(websocket_urlpatterns),
})
```

11.2 Notification Consumer (simplified)

python

```python
# apps/notifications/consumers.py
import json
from channels.generic.websocket import AsyncWebsocketConsumer

class NotificationConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        user = self.scope['user']
        if not user.is_authenticated:
            await self.close()
            return

        self.group_name = f'notifications_{user.id}'
        await self.channel_layer.group_add(self.group_name, self.channel_name)
        await self.accept()

    async def disconnect(self, close_code):
        await self.channel_layer.group_discard(self.group_name, self.channel_na

    async def notification_message(self, event):
        await self.send(text_data=json.dumps(event['notification']))
```

---

12. Hosting and Deployment

12.1 Frontend (Vercel)

```
vercel.json:
{
```

```
  "framework": "nextjs",
  "buildCommand": "npm run build",
  "devCommand": "npm run dev",
  "installCommand": "npm install",
  "env": {
    "NEXT_PUBLIC_API_URL": "@straep_api_url",
    "NEXT_PUBLIC_PAYSTACK_PUBLIC_KEY": "@paystack_public_key"
  }
}
```

Environment variables set in the Vercel dashboard per environment (preview, production).

12.2 Backend (Railway)

Railway services for production:

Service	Image/Config
`api`	Django app, `gunicorn config.wsgi:application --workers 2 --threads 4`
`celery-worker`	Same Django image, `celery -A config worker --loglevel=info --concurrency 4`
`celery-beat`	Same Django image, `celery -A config beat --loglevel=info`
`postgres`	Railway PostgreSQL plugin
`redis`	Railway Redis plugin

12.3 Production Gunicorn Config

python

```python
# gunicorn.conf.py
workers = 2
worker_class = 'gevent'  # or 'sync'
threads = 4
timeout = 30
keepalive = 5
max_requests = 1000
max_requests_jitter = 50
bind = '0.0.0.0:8000'
accesslog = '-'
errorlog = '-'
loglevel = 'info'
```

12.4 CI/CD (GitHub Actions)

yaml

```yaml
# .github/workflows/deploy.yml
name: Deploy

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v4
        with: { python-version: '3.12' }
      - run: pip install -r requirements.txt
      - run: python manage.py test --settings=config.settings.test

  deploy-backend:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to Railway
        run: railway up --service api
        env:
          RAILWAY_TOKEN: ${{ secrets.RAILWAY_TOKEN }}

  deploy-frontend:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to Vercel
        run: vercel --prod --token=${{ secrets.VERCEL_TOKEN }}
```

---

13. Environment Configuration

13.1 Required Environment Variables

bash

```
# Django Backend (.env)

# Core
DJANGO_SECRET_KEY=your-secret-key-here
DJANGO_DEBUG=False
DJANGO_ALLOWED_HOSTS=api.straep.com

# Database
DB_NAME=straep_prod
DB_USER=straep_user
DB_PASSWORD=secure-password
DB_HOST=postgres.railway.internal
DB_PORT=5432

# Redis
REDIS_URL=redis://redis.railway.internal:6379/0

# JWT
JWT_SECRET_KEY=your-jwt-secret-key

# Cloudinary
CLOUDINARY_CLOUD_NAME=straep
CLOUDINARY_API_KEY=your-api-key
CLOUDINARY_API_SECRET=your-api-secret

# Paystack
PAYSTACK_SECRET_KEY=sk_live_your_key
PAYSTACK_PUBLIC_KEY=pk_live_your_key

# Stripe (Phase 2)
STRIPE_SECRET_KEY=sk_live_your_key
STRIPE_WEBHOOK_SECRET=whsec_your_secret

# Resend
RESEND_API_KEY=re_your_api_key

# CORS
CORS_ALLOWED_ORIGINS=https://straep.com,https://www.straep.com

# Frontend URL (for email links)
FRONTEND_URL=https://straep.com
```

bash

```
# Next.js Frontend (.env.local)

NEXT_PUBLIC_API_URL=https://api.straep.com/v1
NEXT_PUBLIC_PAYSTACK_PUBLIC_KEY=pk_live_your_key
NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME=straep
NEXT_PUBLIC_APP_URL=https://straep.com
```

14. Project Structure

14.1 Monorepo Structure

```
straep/                          ← Root monorepo
├── apps/
│   ├── web/                     ← Next.js frontend
│   └── backend/                 ← Django backend
├── packages/
│   └── types/                   ← Shared TypeScript types (optional)
├── docs/                        ← This documentation
├── .github/workflows/           ← CI/CD
├── docker-compose.yml           ← Local development
├── package.json                 ← Root package.json (workspace config)
└── README.md
```

14.2 Docker Compose for Local Development

yaml

```yaml
# docker-compose.yml
version: '3.8'

services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: straep_dev
      POSTGRES_USER: straep
      POSTGRES_PASSWORD: dev_password
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

  backend:
    build: ./apps/backend
    command: python manage.py runserver 0.0.0.0:8000
    volumes:
      - ./apps/backend:/app
    ports:
      - "8000:8000"
    environment:
      - DJANGO_SETTINGS_MODULE=config.settings.development
      - DB_HOST=postgres
      - REDIS_URL=redis://redis:6379/0
    depends_on:
      - postgres
      - redis

  celery:
    build: ./apps/backend
    command: celery -A config worker --loglevel=info
    volumes:
      - ./apps/backend:/app
    environment:
      - DJANGO_SETTINGS_MODULE=config.settings.development
      - DB_HOST=postgres
      - REDIS_URL=redis://redis:6379/0
    depends_on:
      - postgres
      - redis

  frontend:
    build: ./apps/web
    command: npm run dev
```

```
    volumes:
      - ./apps/web:/app
      - /app/node_modules
    ports:
      - "3000:3000"
    environment:
      - NEXT_PUBLIC_API_URL=http://localhost:8000/v1

volumes:
  postgres_data:
```

---

15. Key Implementation Patterns

15.1 Service Layer Pattern

Keep views thin. All business logic goes in a `services.py` file per app.

```python
```

```python
# apps/inquiries/services.py
from django.db import transaction
from tasks.email import send_inquiry_notification_email
from tasks.notifications import create_notification

def submit_inquiry(brand_page, sender_data: dict, product_id=None) -> Inquiry:
    with transaction.atomic():
        inquiry = Inquiry.objects.create(
            brand_page=brand_page,
            sender_name=sender_data['sender_name'],
            message=sender_data['message'],
            product_interest=sender_data.get('product_interest'),
            product_id=product_id,
            contact_preference=sender_data.get('contact_preference', 'straep_me
        )

        # Increment denormalised counter
        brand_page.inquiry_count = F('inquiry_count') + 1
        brand_page.save(update_fields=['inquiry_count'])

    # Async tasks (outside transaction to avoid holding connection)
    send_inquiry_notification_email.delay(str(inquiry.id))
    create_notification.delay(
        recipient_id=str(brand_page.user_id),
        type='new_inquiry',
        entity_type='inquiry',
        entity_id=str(inquiry.id),
        title='New inquiry received',
        body=f'{inquiry.sender_name} sent you an inquiry',
        action_url=f'/dashboard/leads/{inquiry.id}',
    )

    return inquiry
```

15.2 Custom Exception Handler

python

```python
# common/exceptions.py
from rest_framework.views import exception_handler
from rest_framework.response import Response

def custom_exception_handler(exc, context):
    response = exception_handler(exc, context)

    if response is not None:
        custom_response = {
            'success': False,
            'error': {
                'code': getattr(exc, 'default_code', 'ERROR').upper(),
                'message': str(exc.detail) if hasattr(exc, 'detail') else str(e
            }
        }

        # Handle validation errors with field-level details
        if hasattr(exc, 'detail') and isinstance(exc.detail, dict):
            custom_response['error']['code'] = 'VALIDATION_ERROR'
            custom_response['error']['message'] = 'Request validation failed.'
            custom_response['error']['details'] = [
                {'field': field, 'message': str(errors[0])}
                for field, errors in exc.detail.items()
            ]

        response.data = custom_response

    return response
```

---

16. Security Implementation

16.1 Security Checklist

- [ ] `DEBUG = False` in production.
- [ ] `SECRET_KEY` is long, random, and stored in environment variables.
- [ ] `ALLOWED_HOSTS` is set correctly.
- [ ] `HTTPS` enforced; HTTP redirects to HTTPS (`SECURE_SSL_REDIRECT = True`).
- [ ] `SECURE_HSTS_SECONDS = 31536000` (1 year HSTS).
- [ ] `SECURE_CONTENT_TYPE_NOSNIFF = True`.
- [ ] `X_FRAME_OPTIONS = 'DENY'`.
- [ ] CORS: only allowed origins can make API requests.
- [ ] CSRF: enabled for any non-API endpoints.
- [ ] Password hashing: Django's default `PBKDF2` or `bcrypt`.
- [ ] Refresh tokens stored as SHA-256 hashes, not plaintext.
- [ ] Paystack webhook signature verified before processing.
- [ ] File upload: MIME type validated server-side (not just extension).

- [ ] File upload: max size enforced server-side.
- [ ] SQL injection: impossible via Django ORM (no raw queries in hot paths).
- [ ] Rate limiting: implemented at the view level via Django middleware or `django-ratelimit`.
- [ ] Admin endpoints: require `is_staff = True`.
- [ ] Admin panel uses 2FA.
- [ ] Audit log: immutable (no UPDATE/DELETE on `admin_audit_log`).

---

17. Performance Optimisation

17.1 Database Query Optimisation

python

```python
# Always use select_related / prefetch_related to avoid N+1 queries

# Feed post list — optimised
posts = Post.objects.filter(
    id__in=post_ids,
    is_published=True,
    deleted_at__isnull=True,
).select_related(
    'author',
    'author__profile',
    'brand_page',
    'product',
).prefetch_related(
    Prefetch(
        'post_likes',
        queryset=PostLike.objects.filter(user=current_user),
        to_attr='current_user_likes',
    )
).order_by('-created_at')
```

17.2 API Response Caching

Use Django's `@cache_page` or custom Redis caching on expensive read endpoints:

python

```python
from django.utils.decorators import method_decorator
from django.views.decorators.cache import cache_page

class BrandPageView(RetrieveAPIView):
    @method_decorator(cache_page(60 * 5))  # 5-minute cache
    def get(self, request, *args, **kwargs):
        return super().get(request, *args, **kwargs)
```

---

18. Testing Strategy

18.1 Backend Tests

```
apps/users/tests/
  test_registration.py      ← Auth flow tests
  test_authentication.py    ← JWT tests
  test_onboarding.py

apps/brands/tests/
  test_brand_page.py        ← Brand page CRUD
  test_editor.py            ← Section editor

apps/inquiries/tests/
  test_inquiry_submission.py
  test_lead_management.py

apps/subscriptions/tests/
  test_plan_gating.py       ← Feature gate tests
  test_webhook.py           ← Paystack webhook handling
```

18.2 Critical Test Cases

python

```
# Test: Free plan product limit
def test_free_plan_product_limit(self):
    user = create_user(account_type='business')
    brand = create_brand_page(user)

    # Create 5 products (Free plan limit)
    for i in range(5):
        create_product(brand)

    # 6th product should fail
    response = self.client.post(
        f'/v1/brands/{brand.id}/products',
        data={'name': 'Product 6', 'price': 1000},
        HTTP_AUTHORIZATION=f'Bearer {get_token(user)}'
    )
    self.assertEqual(response.status_code, 403)
    self.assertEqual(response.json()['error']['code'], 'PLAN_REQUIRED')

# Test: Webhook idempotency
def test_paystack_webhook_idempotency(self):
    # Send the same webhook twice (Paystack retries)
    payload = build_charge_success_payload(reference='STR-TEST-123')

    response1 = self.client.post('/v1/subscriptions/webhook', payload)
    response2 = self.client.post('/v1/subscriptions/webhook', payload)

    self.assertEqual(response1.status_code, 200)
    self.assertEqual(response2.status_code, 200)

    # Subscription should only be activated once
    subs = Subscription.objects.filter(user=self.user)
    self.assertEqual(subs.count(), 1)
```

---

19. Development Setup Guide

19.1 Prerequisites

bash

```
# Required tools
Node.js 20+
Python 3.12+
Docker Desktop (for local postgres and redis)
Git
```

19.2 Local Setup (Step by Step)

bash

```
# 1. Clone the repository
git clone https://github.com/your-org/straep.git
cd straep

# 2. Start Docker services
docker-compose up -d postgres redis

# 3. Backend setup
cd apps/backend
python -m venv venv
source venv/bin/activate  # Windows: venv\Scripts\activate
pip install -r requirements.txt

# Copy and configure environment
cp .env.example .env
# Edit .env with your values

# Run migrations
python manage.py migrate

# Seed initial data (categories, subscription plans)
python manage.py loaddata fixtures/categories.json
python manage.py loaddata fixtures/subscription_plans.json

# Create a superuser
python manage.py createsuperuser

# Run the dev server
python manage.py runserver

# In a separate terminal: run Celery worker
celery -A config worker --loglevel=info

# 4. Frontend setup
cd apps/web
npm install
cp .env.local.example .env.local
# Edit .env.local
npm run dev
```

19.3 Useful Management Commands

bash

```
# Reset and reseed the database (development only)
python manage.py flush --no-input
python manage.py migrate
python manage.py loaddata fixtures/categories.json
python manage.py loaddata fixtures/subscription_plans.json
python manage.py create_sample_data  # custom command for dev data

# Generate a new secret key
python -c "from django.core.management.utils import get_random_secret_key; prin

# Run tests with coverage
coverage run manage.py test
coverage report
coverage html

# Check for security issues
python manage.py check --deploy
```

End of Tech Stack and Implementation Guide

Next document: 10-Development-Roadmap-and-Sprint-Plan.md